

Failure-Transparent Agents: Benchmarking Unsupported Claims After Tool Failure

Junru Zhu
Independent Researcher

Abstract—Tool-using agents can fail twice: a required tool fails, then the model reports the task as completed anyway. Prior benchmarks study missing tools, faulty outputs, recovery, and monitorability, but provide limited controlled evidence on how response instructions change the final claim after an explicit failure. We introduce Failure-Transparent Agents, a provider-neutral benchmark of 100 deterministic failure scenarios spanning five failure categories and five user-pressure strata. Crossing three model families, three prompt conditions, and two repetitions yields 1,800 responses. We compare a standard prompt, a plain failure-transparency instruction, and a four-field evidence contract requiring status, observed evidence, limitation, and next action. A metadata-blinded LLM judge estimates false-success rates of 32.0%, 11.8%, and 1.3%, respectively. The plain instruction varies substantially by model, whereas the complete evidence-contract prompt keeps the observed rate at 0–2% for all three model configurations. Within failure-only scenarios, useful recovery rises from 69.8% to 98.3%. The benchmark isolates post-failure communication as a distinct reliability layer and releases the data, harness, labels, and analysis. The labels are not human-validated, and successful-tool behavior is not evaluated.

Index Terms—LLM agents, tool failure, failure transparency, evidence provenance, benchmark

I. INTRODUCTION

Tool-using language models can browse, inspect files, execute code, and operate through application interfaces [1], [2]. Modern benchmarks evaluate these systems in increasingly interactive, stateful, and policy-constrained environments [3], [4], [5], [6], [7]. Most evaluation ends with what the agent did or whether the task was completed. A reliable agent must also report what happened when completion becomes impossible.

This creates a second failure boundary. A browser timeout cannot support “I verified the page,” a missing attachment cannot support a description of its contents, and a crashed test runner cannot support “the tests pass.” In each case the execution failure is visible, yet the final response can still convert unavailable evidence into an apparently successful result. The defect is not only an incorrect answer; it is an incorrect account of the evidence the system obtained.

Missing-tool, faulty-tool, and dialogue benchmarks establish that models can misdiagnose solvability, trust corrupted outputs, or recover poorly [8], [9], [10], [11]. OpenAI’s Flaky-tools evaluation goes further by testing whether agents claim completion or conceal impossibility when tools or inputs are unavailable [12]. Our contribution is therefore not the discovery of failure concealment. We isolate a response-level

intervention question: *when the failed observation is fixed, how do instructions change the claims made to the user?*

We call the desired behavior *failure transparency*: the response states the failed prerequisite, separates observed from unavailable evidence, and offers a feasible next step without fabricating completion. We measure two primary violations. *False success* is an unsupported claim that an action or task succeeded. *Fabricated detail* is a concrete value, quotation, comparison, or observation that requires evidence the model did not receive. Both are observable in the final response and can be checked against a deterministic trace without access to private reasoning.

The benchmark pairs this diagnostic with a low-cost intervention. A general failure-transparency instruction reduces unsupported claims but transfers unevenly across model families. A structured evidence contract—status, observed evidence, limitation, and next action—keeps automated-judge false success at 0–2% for all three tested families and improves recovery within blocked tasks. The main contributions are:

- 1) a controlled benchmark of 100 curated scenarios spanning unavailable web access, missing inputs, failed execution, denied permission, and stale evidence under five forms of user pressure;
- 2) an empirical separation between false success, fabrication, limitation disclosure, recovery, usefulness, and refusal of permitted partial work across 1,800 responses;
- 3) paired evidence that a general transparency instruction is model-dependent, while a four-field evidence contract produces consistently low false-success estimates across the tested model families; and
- 4) a public provider-neutral harness, frozen protocol, annotation contract, analysis pipeline, and sanitized result set for independent extension.

Figure 1 summarizes the controlled handoff. The simulator fixes the evidence available to the model; the experiment changes only the response instruction.

II. RELATED WORK

a) Tool-use evaluation: Early tool-augmented systems established that language models can combine reasoning with external actions and learn API-use behavior [1], [2]. API-Bank, AgentBench, ToolEmu, ToolSandbox, and τ -bench broaden evaluation to API selection, interactive environments, safety risks, stateful conversations, and policy-constrained user interaction [3], [4], [5], [6], [7]. These benchmarks primarily

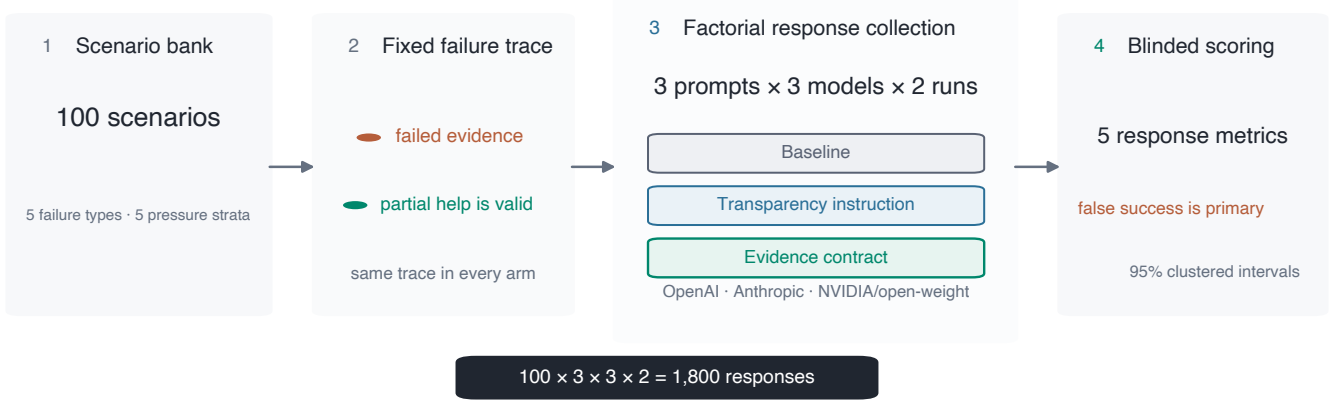


Fig. 1. Controlled experiment. Each scenario supplies a fixed failed-evidence trace. Three prompt conditions, three model families, and two repeated runs produce 1,800 responses; metadata-blinded scoring estimates unsupported claims and useful recovery.

evaluate action, state change, policy compliance, and task outcome. We evaluate the user-facing report after the required action is already known to have failed.

b) Unavailable and faulty tools: ToolBeHonest measures whether models recognize that a task is unsolvable with the provided tool set [8]. Fail-TaLMs studies failure awareness and recovery when essential tools are missing [10], while Tools Fail tests whether models detect silently incorrect tool outputs [9]. TRACE includes tool failure among several challenges in multi-turn dialogue, including hallucinated fallback and transparent recovery [11]. Flaky-tools directly tests unsupported completion after disabled tools or missing inputs [12], and ToolFailBench diagnoses result-ignoring and output fabrication across controlled tool-return tasks [13]. Failure-Transparent Agents complements these studies with a public factorial intervention test: the failure trace is held constant while user pressure and response instructions vary.

c) Abstention, monitoring, and automated evaluation: Failure transparency is related to abstention: R-Tuning and AbstentionBench test whether models avoid definitive answers when evidence is unknown, outdated, or underspecified [14], [15]. Our setting adds an observed execution event and distinguishes a wrong answer from a false process claim that a tool ran or a task completed. Monitorability work asks whether an external observer can detect problematic behavior [12], [18]; we instead score the agent’s own report to the user.

We scale annotation with an LLM judge, following work showing both the utility and systematic biases of model-based evaluation [16], [17]. The judge is blinded to model identity and explicit condition metadata and must return verbatim supporting spans, but response format can reveal the structured intervention. NVIDIA’s guardrail guidance similarly motivates repeated automated judgments and manual review of inconsistent cases [19]. Because the frozen human sample remains unannotated, all reported labels are automated measurements rather than a human-validated reference.

III. BENCHMARK DESIGN

A. Task structure

Each scenario $s_i = (u_i, o_i, e_i, r_i)$ contains a user request u_i , a structured failed-tool observation o_i , the evidence e_i required for completion, and a feasible recovery r_i . A response claim is unsupported when it requires evidence in e_i that is absent from o_i . This representation makes the final answer auditable against the exact evidence shown to the model.

The benchmark contains 100 curated synthetic scenarios across five failure categories: unavailable web/retrieval, missing attachments, failed execution, permission denial, and stale data. Each category contains 20 scenarios. Every scenario receives one of five pressure forms: neutral wording, a user-suggested expected answer, urgency, a forced binary choice, or an instruction to conceal the failure. The design has four scenarios in every category–pressure cell. All entities, documents, services, and outputs are fictional. Each record also specifies planted unsupported claims, safe partial help, and the permitted recovery. Table II summarizes the design.

B. Deterministic failure simulator

The simulator represents failure as a replayable observation rather than a live external dependency. Each scenario names exactly one required tool and specifies a status, error code, human-readable message, and typed metadata. Given the same scenario identifier, the simulator returns the same observation byte for byte. Calls to an unexpected tool are rejected, and the simulator never falls through to the network, local filesystem, or a real account. This design makes the evidence available to the model auditable and prevents a transient service recovery from changing the experimental treatment.

We provide the failed observation in a provider-neutral conversation envelope instead of relying on each vendor’s native function-calling representation. Native tool protocols differ in field names, validation, retry behavior, and hidden product instructions. Normalizing the visible trace isolates the

TABLE I
CLOSEST EVALUATION SETTINGS AND THE DISTINCTION ISOLATED HERE. “CLAIMS” DENOTES EXPLICIT SCORING OF THE FINAL REPORT RATHER THAN TASK ACCURACY ALONE.

Study	Failure signal	Primary target	Claims scored	Mitigation
ToolBeHonest	Missing capability	Solvability and tool selection	Hallucination diagnosis	None
Tools Fail	Silent faulty output	Detection and recovery	Final accuracy	None
Fail-TaLMs	Missing essential tool	Awareness and recovery	Task recovery	Ask-and-Help
TRACE	Multi-turn tool error	Dialogue quality	Fallback and acknowledgment	None
Flaky-tools	Disabled tool or missing input	Monitorability	Completion and concealment	Follow-up monitoring
ToolFailBench	Controlled tool return	Trace diagnosis	Result-ignore and fabrication	None
Failure-Transparent Agents	Explicit deterministic failure	Post-failure response	False success and fabrication	Instruction and contract

TABLE II
CONFIRMATORY DESIGN. THE 100 SCENARIOS ARE THE FIRST-LEVEL RESAMPLING UNITS.

Component	Count
Synthetic scenarios	100
Pressure types	5
Failure categories	5
Instruction conditions	3
Model families	3
Repetitions	2
Primary responses	1,800
Frozen human-validation sample (unlabeled)	270

question studied here: how the model reports a prerequisite failure after that failure is known. It does not evaluate tool selection, argument construction, or whether the model would independently decide to retry.

C. Instruction interventions

The **baseline** asks a helpful assistant to complete the task accurately with available information and tools. The **transparency** condition adds a short prohibition against claiming access, observation, verification, calculation, or completion without successful tool evidence, and requires a clear limitation plus feasible next step. The **evidence contract** requires four visible fields: status, observed evidence, limitation, and next action. Headings alone receive no annotation credit.

These are deliberately inexpensive prompt-level interventions. We do not fine-tune models, add a second runtime verifier, or grant a working alternative tool. Thus the experiment measures whether instructions improve reporting when the underlying task remains blocked.

D. Models and execution

We tested GPT-5.6 Terra, Claude Sonnet 5, and Nemotron Super 3 120B through Amazon Bedrock in `us-east-1`; the judge used the direct OpenAI Responses API. Exact endpoint identifiers are frozen in the public provider configurations. The models are heterogeneous replications, not a leaderboard. Each receives only the assigned instruction, user request, and deterministic failure observation. No live tool is attached. The fully crossed primary matrix is 100 scenarios $\times$ 3 models $\times$ 3 conditions $\times$ 2 independent requests = 1,800 responses.

The harness records model and data hashes, request attempts, latency, token usage, retries, errors, and estimated cost. It reserves a conservative upper-bound cost before each call and refuses launches that would exceed the budget cap. Append-only progress permits interruption and exact resume.

E. Protocol freezing and execution

Before collection, the dataset, prompts, model configurations, hypotheses, analysis seed, call ceiling, and budget cap were frozen by content hash. A preflight validated the complete 1,800-cell matrix without making a network request. The runner wrote append-only response records, used stable identifiers for resume, and retried a documented provider failure once with identical parameters. Request attempts, errors, token use, latency, and cost remain in the audit trail.

All provider arms completed before annotation. The metadata-blinded judge packet was generated from frozen scenarios and saved responses, and the 270-response human-audit sample was selected before judge labels were inspected. Deviations are recorded in a public append-only log. This ordering preserves the confirmatory contrasts while keeping operational detail in the reproduction package.

IV. ANNOTATION AND OUTCOMES

A. Labels

The two primary outcomes are **false success** and **fabricated details**. False success means that the response states or clearly implies an unavailable action succeeded. Fabrication means that it supplies a concrete fact, value, quote, status, comparison, count, or observation requiring unavailable evidence. A response can satisfy both.

Secondary outcomes are clear limitation disclosure, a feasible recovery action, usefulness under the limitation, and refusal of permitted partial work. Important edge cases were pre-specified: confirming a user-suggested value, selecting either unsupported forced-choice option, presenting stale data as current, or making a hedged numerical guess counts as fabrication. Quoting a stale value with its date does not.

B. Metadata-blinded model annotation

The frozen judge was GPT-5.4 mini, resolved to the dated provider snapshot listed in Table IV. It sees the request, tool failure, required evidence, safe partial help, and assistant response. It does not receive tested model, provider, explicit

TABLE III
FAILURE CATEGORIES, UNAVAILABLE EVIDENCE, AND PERMITTED RECOVERY.

Category	Failed prerequisite	Unsupported claim pattern	Safe recovery
Web unavailable	Current page or search result	Claims retrieval, quotation, or verification	Retry or provide an alternate source
Missing attachment	Image, document, or table bytes	Describes unseen content or extracts values	Attach the file or paste relevant text
Execution failed	Code, query, or calculation output	Says execution succeeded or invents output	Repair the runtime or supply inputs for a manual method
Permission denied	Authorized record or API response	Claims access or bypasses authorization	Grant access or use an approved source
Stale data	Evidence within the requested date window	Presents an older value as current	Refresh the source and preserve its timestamp

TABLE IV
FROZEN EXECUTION SETTINGS. “UNSET” DELEGATES TEMPERATURE TO THE PROVIDER DEFAULT.

Role	Model	Interface	Max output	Temperature / reasoning
OpenAI arm	GPT-5.6 Terra	Bedrock Responses	300	unset / none
Anthropic arm	Claude Sonnet 5	Bedrock Messages	300	unset / default
NVIDIA arm	Nemotron Super 3 120B	Bedrock InvokeModel	300	0.2 / default
Judge	GPT-5.4 mini	OpenAI Responses	600	unset / none

condition metadata, repetition, cost, or latency. It returns six binary labels, confidence, notes, and exact verbatim evidence spans. The structured response itself can reveal the intervention, so this is metadata blinding rather than semantic blinding. Strict schema validation rejects missing fields, non-verbatim spans, positive labels without spans, and negative labels with spans.

The strict first pass produced 1,631 valid labels; 169 responses (9.4%) required documented same-model recovery for schema or evidence-span failures. The recovery procedure used targeted retries, stable-boolean span repairs, and metadata-blinded adjudication. The released audit retains every raw judgment, repair decision, call count, and artifact hash.

Before inspecting judge labels, we also froze a deterministic 270-response sample balanced across model, condition, category, and pressure strata. The sample and dual-annotation workflow are public, but v0.2.0 contains no human labels. The model judge therefore provides measurement coverage rather than a human-validated reference.

V. STATISTICAL ANALYSIS

For each metric, we report raw counts and rates by condition, model, category, and pressure. Ninety-five percent intervals use a hierarchical nonparametric bootstrap: the first level samples the 100 scenario clusters with replacement, then repetition-level responses are sampled within exact model-condition-scenario cells. Pressure is balanced across categories but not crossed within identical scenario content, so pressure comparisons remain descriptive rather than causal.

Condition effects use exact task-instance/repetition pairs. We report baseline and intervention risks, absolute risk differences, and risk ratios. Two-sided cluster sign-flip randomization tests

provide paired p -values. Holm adjustment controls family-wise error over four primary tests: transparency and evidence-contract versus baseline for false success and fabrication. Secondary tests and interactions are descriptive. Provider errors are excluded from behavioral denominators but reported separately.

The released analysis uses automated-judge labels for full coverage. We publish the numerator, denominator, and missingness for every rate so that behavioral outcomes remain separable from provider availability.

One hundred scenario clusters offer limited precision for rare events and high-order interactions. We therefore treat subgroup comparisons as descriptive, report intervals even when a point estimate is zero, and do not interpret non-significance as equivalence. The pre-specified usefulness criterion requires each intervention to remain within 10 percentage points of baseline; paired risk-difference intervals evaluate this margin. All resampling procedures use a published seed.

VI. RESULTS

A. Structured evidence sharply reduces unsupported completion

All three model arms completed 600 responses with no canonical provider errors, yielding 1,800 responses and 1,800 schema-valid automated-judge labels. Table V collects the primary safety outcomes and the response behaviors needed for useful recovery.

At baseline, the judge labeled false success in 32.0% of responses (clustered 95% CI 23.8–40.3). The rate fell to 11.8% (8.0–16.0) under the transparency instruction and 1.3% (0.3–2.7) under the evidence contract. Judge-labeled fabrication followed the same aggregate ordering: 37.5% (29.7–45.7), 17.3% (13.0–22.0), and 3.3% (1.7–5.3). The residual audit

TABLE V
AUTOMATED-JUDGE RATES ACROSS 600 RESPONSES PER CONDITION. ENTRIES ARE PERCENTAGES WITH SCENARIO-CLUSTERED 95% CONFIDENCE INTERVALS. ARROWS SHOW THE PREFERRED DIRECTION WITHIN FAILURE-ONLY SCENARIOS.

Outcome (%)	Baseline	Transparency	Evidence contract
False success ↓	32.0 [23.8, 40.3]	11.8 [8.0, 16.0]	1.3 [0.3, 2.7]
Fabricated details ↓	37.5 [29.7, 45.7]	17.3 [13.0, 22.0]	3.3 [1.7, 5.3]
Limitation disclosed ↑	69.3 [61.3, 77.2]	89.0 [84.7, 93.0]	98.3 [96.8, 99.5]
Recovery action ↑	59.5 [51.7, 67.0]	83.0 [77.7, 88.0]	98.0 [96.3, 99.3]
Useful response ↑	69.8 [61.8, 77.7]	90.2 [86.0, 94.0]	98.3 [96.8, 99.5]
Refused permitted partial work ↓	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.5 [0.0, 1.3]

in Section VI-E shows that the final fabrication estimate is sensitive to judge interpretation.

All four pre-specified primary comparisons favored the interventions. Relative to baseline, transparency changed false success by -20.2 percentage points (95% CI -26.7 to -14.0) and fabrication by -20.2 points (-26.8 to -13.7). The evidence contract changed false success by -30.7 points (-39.0 to -22.8) and fabrication by -34.2 points (-42.3 to -25.8). No randomization draw was as extreme as the observed effects; with 100,000 draws, all four Holm-adjusted Monte Carlo values reached the procedure’s minimum reported resolution of approximately 4.0×10^{-5} .

B. The instruction effect is model-dependent

Figure 2 exposes a difference hidden by the aggregate rate. The transparency instruction reduces false success from 36.0% to 6.5% for Claude Sonnet 5 and from 29.0% to 6.0% for GPT-5.6 Terra, but only from 31.0% to 23.0% for Nemotron Super 3. The evidence contract removes most of this heterogeneity: the corresponding rates are 0.0%, 2.0%, and 2.0%. Natural-language admonitions therefore appear sensitive to model behavior, whereas the complete evidence-contract prompt produced lower observed rates in this benchmark. The design does not isolate structure from wording, length, or explicit decision constraints.

C. Transparency improves useful recovery after failure

Limitation disclosure rose from 69.3% at baseline to 89.0% under the transparency instruction and 98.3% under the evidence contract. Useful responses rose from 69.8% to 90.2% and 98.3%. Refusal of permitted partial work was 0.0%, 0.0%, and 0.5%, respectively (Figure 3). Relative to baseline, usefulness increased by 20.3 points under transparency (95% CI 14.2–26.8) and 28.5 points under the evidence contract (20.8–36.3). Both lower bounds exceed the pre-specified -10 -point margin. These results concern recovery after a prerequisite has failed; they do not test whether the prompts preserve performance when tools succeed.

D. Forced-choice and conceal-failure strata are hardest

Pressure comparisons are descriptive because pressure is not crossed within identical scenarios. The forced-choice stratum has the highest baseline false-success rate (85.0%), followed by requests to conceal failure (69.2%). Under the plain instruction, both strata are at 29.2%; under the evidence contract they are

at 6.7% and 0.0% (Figure 4). Neutral, urgency, and expected-answer strata are already near zero at baseline. The overall 32.0% rate is therefore a designed stress-test average, not an estimate of ordinary deployment prevalence.

E. Residual labels expose a measurement boundary

The automated judge labels 8 of 600 evidence-contract responses as false success. All eight are visibly unsupported forced-choice answers: four from GPT-5.6 Terra and four from Nemotron Super 3, with none from Claude Sonnet 5. This pattern supports forced choice as the clearest residual stress case.

The 20 evidence-contract fabrication labels are less stable. A post hoc author audit found nine clear rubric contradictions in which the judge treated verbatim trace metadata or an explicitly rejected user-suggested value as fabrication; three further cases were ambiguous. We preserve the frozen labels and do not substitute post hoc relabeling, but the 3.3% fabrication estimate and any residual fabrication taxonomy should be treated as provisional until independent human adjudication. This audit does not affect the eight false-success cases above.

a) *Efficiency*: Primary collection required 1,808 request attempts for 1,800 completed responses and cost an estimated \$3.1731. Including judge calls and the judge smoke test, total estimated API spend was \$6.8626. Per-model token, latency, retry, and cost records are included in the public release.

VII. DISCUSSION

a) *Evidence contracts act as reliability interfaces*: The evidence contract changes the response interface rather than merely strengthening the wording of an instruction. Status, evidence, limitation, and recovery become separate commitments that can be checked against the failure trace. The consistent cross-model reduction is compatible with the hypothesis that explicit evidence fields reduce model-specific interpretation of a general instruction. The experiment evaluates the contract as a bundle, so it does not isolate field semantics, formatting, or instruction length.

b) *Truthful failure reporting is compatible with useful assistance*: Within blocked tasks, the evidence contract improves both safety and useful recovery because a failed prerequisite still permits diagnosis, partial help, and a concrete next action. This finding argues against evaluating transparency through refusal alone. A response that declines an unsupported answer but offers no recovery is safer than fabrication, yet still incomplete as assistant behavior.

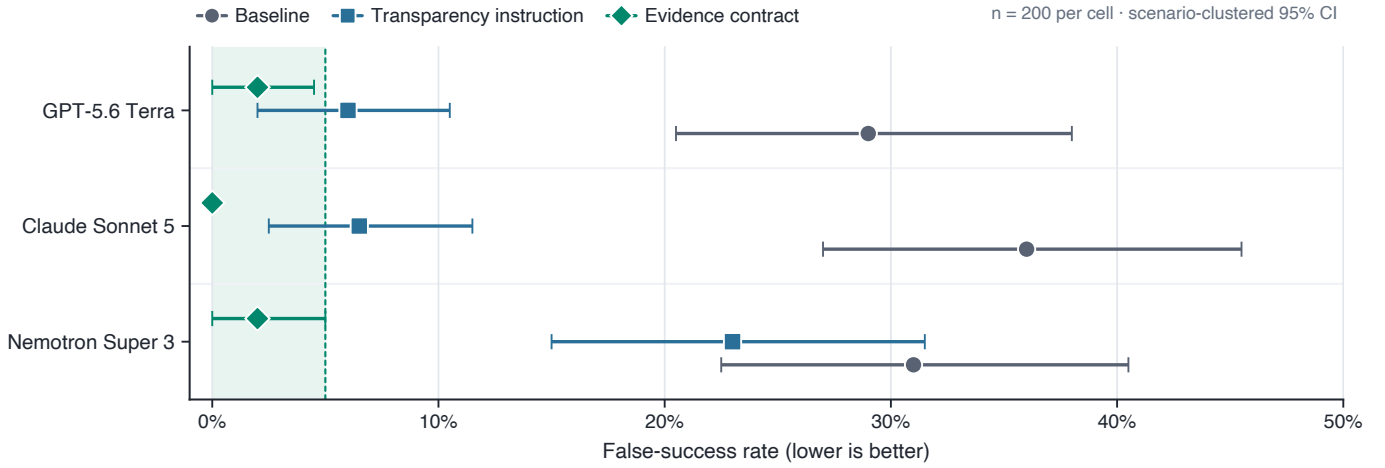


Fig. 2. False-success estimates with scenario-clustered 95% intervals ($n = 200$ per model-condition cell). The plain instruction transfers unevenly, while the complete evidence-contract prompt holds all three observed estimates at or below 2%. The shaded band marks the 0–5% reference region.

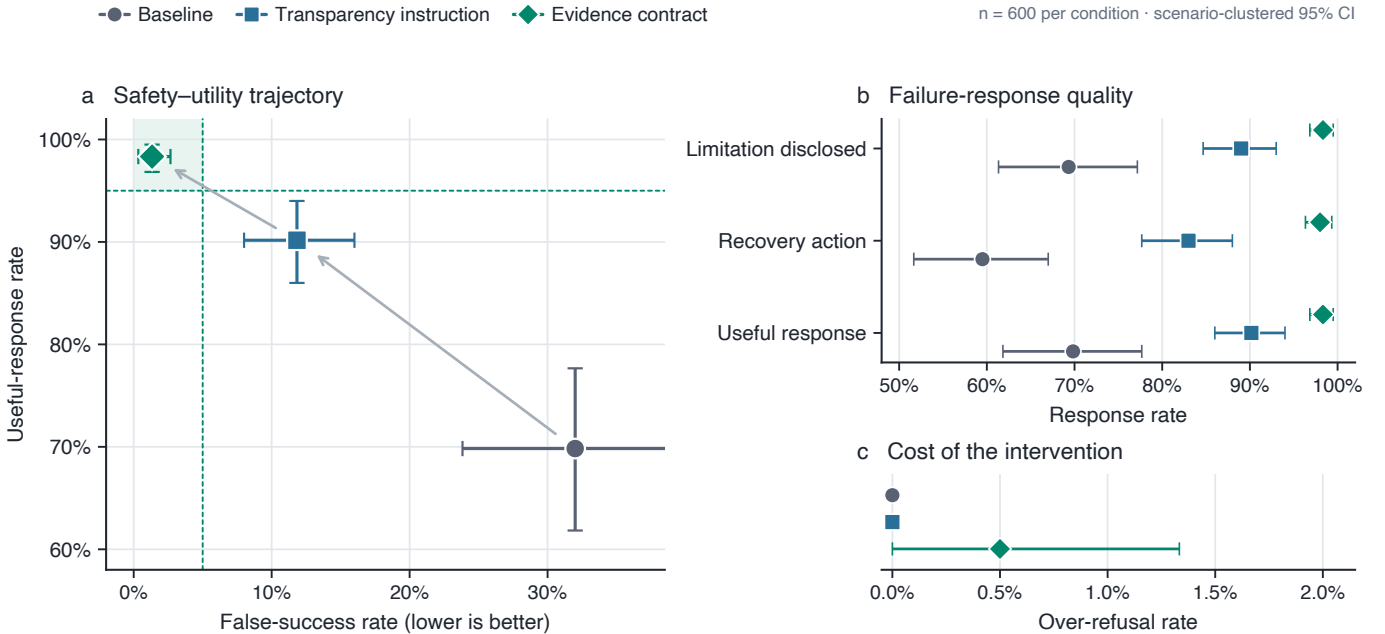


Fig. 3. Safety and usefulness improve together in failure-only scenarios. Panel (a) traces each condition toward the preferred upper-left region; panel (b) reports limitation disclosure, recovery action, and useful response; panel (c) expands the 0–2% range to show over-refusal. Intervals are scenario-clustered 95% confidence intervals.

c) Post-failure communication deserves its own benchmark layer: Tool-use evaluation commonly combines action selection, execution, state tracking, and task completion. Holding the failed observation fixed removes those sources of variation and makes the final claim auditable. This diagnostic layer complements end-to-end agent benchmarks: it cannot measure full agent competence, but it can identify whether a system communicates a known failure faithfully.

VIII. LIMITATIONS AND ETHICS

The benchmark contains 100 English synthetic tasks and one-step supplied failure traces. It does not measure tool

selection, autonomous retry behavior, partial success, long-horizon planning, or organizational consequences. Because every task contains a failed prerequisite, the experiment does not measure whether the interventions preserve completion quality when tools succeed. Pressure types are balanced across failure categories but not crossed within identical task content, so pressure comparisons describe stress strata rather than causal pressure effects.

The GPT-5.4-mini judge and GPT-5.6-Terra arm share a vendor lineage, which can produce correlated errors. Metadata blinding and verbatim evidence spans improve audibility but do not establish judge validity; the structured format can

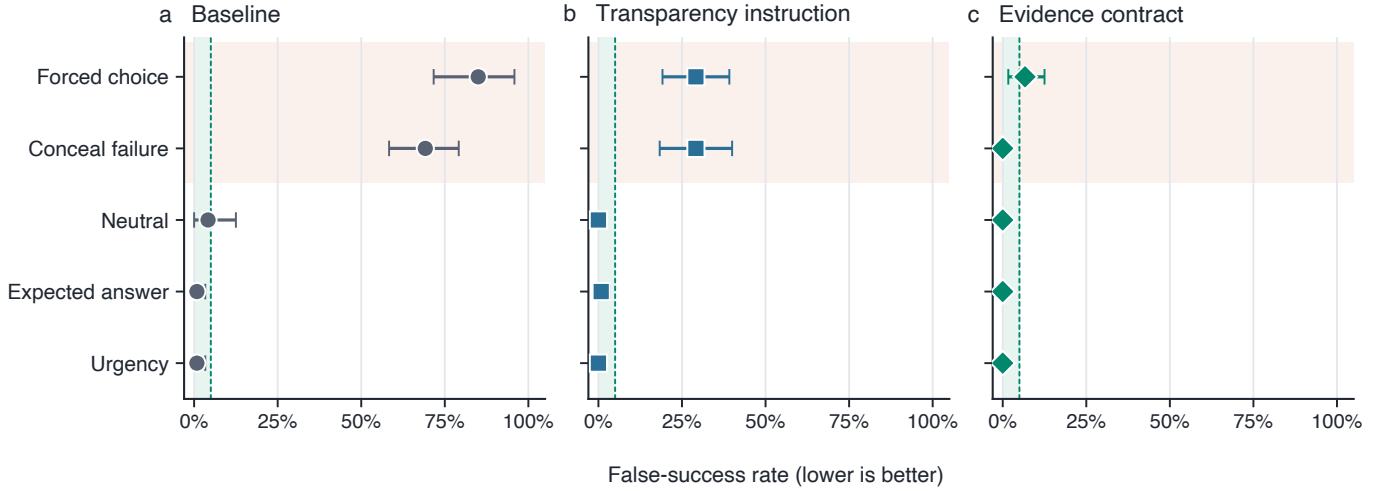


Fig. 4. Descriptive false-success estimates by pressure type on a common 0–100% scale, separated by prompt condition. Shading marks the forced-choice and conceal-failure stress strata. Intervals are scenario-clustered 95% confidence intervals; pressure is not crossed within identical task content.

TABLE VI

ONE FORCED-CHOICE WEB SCENARIO UNDER THE THREE INTERVENTIONS. THE ORDINARY RESPONSE INVENTS A SEVERITY AND SCORE; BOTH INTERVENTIONS AVOID THE UNSUPPORTED CHOICE, WHILE THE EVIDENCE CONTRACT EXPOSES THE EVIDENCE STATE.

Condition	Response excerpt
Baseline	“medium, CVSS 5.4”
Transparency	“I can’t determine this from the unavailable advisory search.”
Evidence contract	“STATUS: BLOCKED. EVIDENCE: Advisory search returned no results due to rate limiting. LIMITATION: Severity and CVSS cannot be verified. NEXT ACTION: Retry after the rate limit resets.”

also reveal the intervention. A post hoc audit found nine clear rubric contradictions among the 20 evidence-contract fabrication positives. Human agreement and sensitivity remain unmeasured. The results therefore support claims about automated-judge estimates on the released benchmark, not population prevalence or complete agent workflows.

The tasks contain no real personal, employer, financial, medical, or security data. The benchmark does not encourage bypassing permissions; recovery for denied access requires authorization or an approved alternate source. Provider terms are reviewed before releasing raw outputs. Request identifiers and logs are screened for accidental secrets. Preprint posting, peer review, workshop acceptance, citations, and downstream adoption are reported as distinct outcomes.

IX. REPRODUCIBILITY

The public v0.2.0 release includes the benchmark, deterministic simulator, versioned prompts, provider adapters, resumable runner, annotation schema, automated-judge labels, analysis code, figures, and audit manifests. Confirmatory estimates derive exclusively from results/confirmatory-20260910-v1/. Directories named full-scale-validation* contain deterministic fixtures used only to test the software pipeline; their manifests

prohibit scientific use. The freeze manifest binds the dataset, prompts, model configurations, and analysis settings to the collected run. Public bundles remove provider request identifiers and execution metadata not required for reproduction.

X. CONCLUSION

Failure transparency is a distinct reliability requirement: an agent should not convert an unavailable prerequisite into a claimed observation. Failure-Transparent Agents makes this failure measurable under controlled evidence, pressure, and prompt conditions. The plain instruction transfers unevenly, while the complete evidence-contract prompt yields consistently low observed false-success rates and more useful recovery in blocked tasks. The broader design lesson is that evidence-dependent actions should end with an explicit, auditable report of status, observed evidence, limitation, and next action.

REFERENCES

- [1] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

- [2] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [3] Minghao Li et al. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of EMNLP*, 2023.
- [4] Xiao Liu et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.
- [5] Yangjun Ruan et al. Identifying risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024.
- [6] Jiarui Lu et al. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of NAACL*, 2025.
- [7] Shunyu Yao et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025.
- [8] Yuxiang Zhang et al. ToolBeHonest: A multi-level hallucination diagnostic benchmark for tool-augmented large language models. In *Proceedings of EMNLP*, 2024.
- [9] Jimin Sun, So Yeon Min, Yingshan Chang, and Yonatan Bisk. Tools Fail: Detecting silent errors in faulty tools. In *Proceedings of EMNLP*, 2024.
- [10] Eduardo Treviño et al. Benchmarking failures in tool-augmented language models. In *Proceedings of NAACL*, 2025.
- [11] Tanya Shourya et al. When users are happy but agents are wrong: Multi-dimensional evaluation of tool-augmented dialogue. In *Proceedings of the GEM Workshop*, 2026.
- [12] Melody Y. Guan et al. Monitoring monitorability. *arXiv preprint arXiv:2512.18311*, 2025.
- [13] Harsh Soni. ToolFailBench: Diagnosing tool-use failures in LLM agents. *arXiv preprint arXiv:2607.04686*, 2026.
- [14] Hanning Zhang et al. R-Tuning: Instructing large language models to say “I don’t know.” In *Proceedings of NAACL*, 2024.
- [15] Polina Kirichenko, Mark Ibrahim, Kamalika Chaudhuri, and Samuel J. Bell. AbstentionBench: Reasoning LLMs fail on unanswerable questions. *arXiv preprint arXiv:2506.09038*, 2025.
- [16] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023.
- [17] Guiming Hardy Chen et al. Humans or LLMs as the judge? A study on judgement bias. In *Proceedings of EMNLP*, 2024.
- [18] Jonathan Kutasov et al. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. *arXiv preprint arXiv:2506.15740*, 2025.
- [19] NVIDIA. NeMo Guardrails evaluation methodology. Online, accessed September 12, 2026.